

Manual de usuario

Este manual explica cómo **compilar** el proyecto y cómo **usar** los tres programas: `assembler1` (una pasada), `assembler2` (dos pasadas) y `minilinker` (enlazador).

1. Requisitos

- Un compilador de C compatible con C11(`gcc` o `clang`).
- `make` .
- Un sistema tipo Unix (Linux, macOS, WSL). En Windows funciona con WSL o MinGW.

No se necesita ninguna biblioteca externa.

2. Compilación

Desde la raíz del proyecto:

```
make
```

Esto genera tres ejecutables en la raíz: `assembler1` , `assembler2` y `minilinker` , y deja los objetos intermedios en `build/` .

Otras órdenes útiles:

```
make clean      # borra binarios y archivos generados
make test       # compila y ejecuta scripts/run_tests.sh
```

También hay scripts equivalentes:

```
bash scripts/build.sh      # = make
bash scripts/run_tests.sh  # ejecuta todos los casos de prueba
bash scripts/clean.sh      # = make clean
```

3. Uso del ensamblador de una pasada

```
./assembler1 <entrada.asm> <prefijo_salida>
```

Ejemplo:

```
./assembler1 examples/test1.asm build/test1
```

Lee el archivo **una sola vez**. Define los símbolos a medida que aparecen, genera el código (parcial para las referencias hacia adelante) y al final aplica los *fixups*: parcha lo que puede resolver localmente y convierte el resto en relokaciones.

Genera:

Archivo	Contenido
build/test1.o	archivo objeto (código en hexadecimal)
build/test1_symbols.txt	tabla de símbolos
build/test1_references.txt	tabla de referencias (fixups y su estado)
build/test1_sections.txt	tabla de secciones (tamaños)
build/test1_relocations.txt	tabla de relokaciones

4. Uso del ensamblador de dos pasadas

```
./assembler2 <entrada.asm> <prefijo_salida>
```

Ejemplo:

```
./assembler2 examples/test1.asm build/test1
```

- **Primera pasada:** construye la tabla de símbolos, calcula direcciones y detecta redefiniciones.
- **Segunda pasada:** genera el código definitivo, resuelve referencias y genera el objeto.

Genera:

Archivo	Pasada	Contenido
build/test1_symbols.txt	1	tabla de símbolos
build/test1_sizes.txt	1	tamaños de cada instrucción/dato
build/test1.o	2	archivo objeto
build/test1_sections.txt	2	tabla de secciones
build/test1_relocations.txt	2	tabla de relokaciones

Los dos ensambladores comparten el mismo núcleo (`asmcore`), por lo que producen exactamente el mismo diseño de secciones y los mismos tamaños.

5. Uso del enlazador (mini linker)

```
./minilinker <mod1.o> <mod2.o> [...] <salida.hex>
```

El **último** argumento es siempre el archivo de salida; los anteriores son los módulos objeto de entrada (uno o más).

Ejemplo completo con dos módulos:

```
./assembler2 examples/mod1.asm build/mod1
./assembler2 examples/mod2.asm build/mod2
./minilinker build/mod1.o build/mod2.o build/program.hex
```

El enlazador:

1. Lee los `.o` con sus tablas de símbolos, secciones y relocalaciones.
2. Fusiona todas las `.text`, todas las `.data` y todas las `.bss`.
3. Resuelve los símbolos `extern` contra los `global` de los otros módulos (detecta duplicados y símbolos no resueltos).
4. Reubica las direcciones (`ABS32` y `REL32`).
5. Escribe el binario final en hexadecimal y un mapa de símbolos.

Genera:

Archivo	Contenido
<code>build/program.hex</code>	imagen final enlazada (hex por secciones)
<code>build/program_map.txt</code>	mapa de símbolos con direcciones finales

6. Sintaxis de entrada admitida

- **Secciones:** `section .text`, `section .data`, `section .bss`.
- **Símbolos:** `global nombre`, `extern nombre`, `nombre equ valor`.
- **Etiquetas:** deben terminar en dos puntos (`etiqueta:`).
- **Datos:** `db`, `dw`, `dd` (con números, símbolos en `dd`, o cadenas en `db`); `resb`, `resw`, `resd` (solo en `.bss`).
- **Registros:** `eax`, `ebx`, `ecx`, `edx`, `esi`, `edi`, `ebp`, `esp`.
- **Números:** decimal (`100`, `-5`) o hexadecimal (`0x1F`).
- **Comentarios:** con `;` hasta el fin de línea.
- **Modos de direccionamiento:** `[1000]`, `[ebp+4]`, `[esi+ecx]`, `[ebx+esi+8]`, `[ebx+ecx*4]`, `[ebx+ecx*4+32]`, `[etiqueta]`.

Programa mínimo de ejemplo:

```
section .data
saludo: db "Hola", 0

section .text
global _start
_start:
    mov eax, 10
    mov ebx, eax
    jmp fin
fin:
    ret
```

7. Mensajes de error

Los errores se imprimen en `stderr` con el formato `archivo:linea: error: descripción`, y el programa termina con código de salida distinto de cero. Ejemplos que el sistema detecta:

- Símbolo indefinido,
- Redefinición de etiqueta o constante,
- `ESP` usado como índice,
- Escala de SIB inválida,
- Datos colocados en `.bss`, o `resb/resw/resd` fuera de `.bss`,
- Combinación de operandos no soportada,
- Símbolo global duplicado o externo no resuelto (en el enlazador).